

Базы данных и SQL

Спроектировать простую реляционную таблицу и понимать строки, колонки, первичный/внешний ключ, ограничения и NULL.

DAY 21

DATABASE

table

row

column

primary key

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Спроектировать простую реляционную таблицу и понимать строки, колонки, первичный/внешний ключ, ограничения и NULL.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример записи и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Командная строка и Linux

Что уже есть в нашей модели.



СЕГОДНЯ

Базы данных и SQL

Вчера: Командная строка и Linux. Сегодня мы добавляем следующий слой: базы данных и sql.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Данные приложения должны храниться структурированно, проверяться ограничениями и связываться между сущностями.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

База данных похожа на аккуратный архив: таблица — шкаф, строка — карточка записи, колонка — поле карточки, primary key — уникальный номер дела.

ПЕРЕВОД В IT

database

организованное хранилище данных с правилами доступа

table

структура строк и колонок одного типа сущностей

row

одна запись таблицы

Слой 1: database • table • row

Сначала смысл, затем профессиональный термин.

**database**

организованное хранилище данных с правилами доступа

**table**

структура строк и колонок одного типа сущностей

**row**

одна запись таблицы

Слой 2: column • primary key • SQL

Сначала смысл, затем профессиональный термин.

**column**

именованное свойство записей таблицы

**primary key**

уникальный идентификатор строки

**SQL**

язык работы с реляционными базами данных

Слой 3: schema • foreign key

Сначала смысл, затем профессиональный термин.

schema

структура и правила организации базы данных

foreign key

ссылка одной таблицы на запись другой

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: constraint • NULL

Сначала смысл, затем профессиональный термин.

constraint

ограничение допустимых данных таблицы

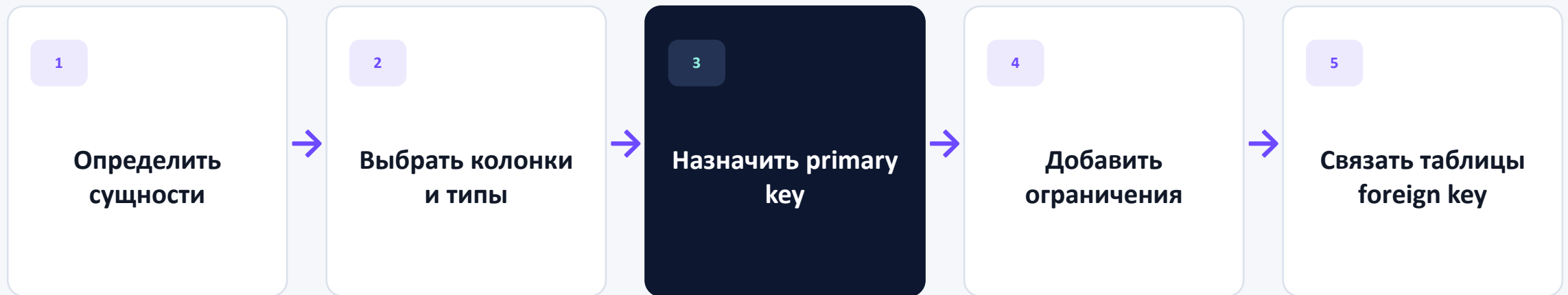
NULL

отсутствующее или неизвестное значение в базе

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.

```
CREATE TABLE courses (  
  id INTEGER PRIMARY KEY,  
  title TEXT NOT NULL,  
  days INTEGER NOT NULL  
);
```

day21_step1

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

CREATE TABLE courses (

Python выполняет эту строку сверху вниз.

id INTEGER PRIMARY KEY,

Python выполняет эту строку сверху вниз.

title TEXT NOT NULL,

Python выполняет эту строку сверху вниз.

days INTEGER NOT NULL

Python выполняет эту строку сверху вниз.

);

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



```
CREATE TABLE courses (  
  id INTEGER PRIMARY KEY,  
  title TEXT NOT NULL,  
  days INTEGER NOT NULL  
);
```

до запуска

РЕЗУЛЬТАТ

Создана таблица courses с тремя колонками

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
INSERT INTO courses (id, title, days)
VALUES (1, 'Python Basics', 30);

SELECT * FROM courses;
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

1 | Python Basics | 30

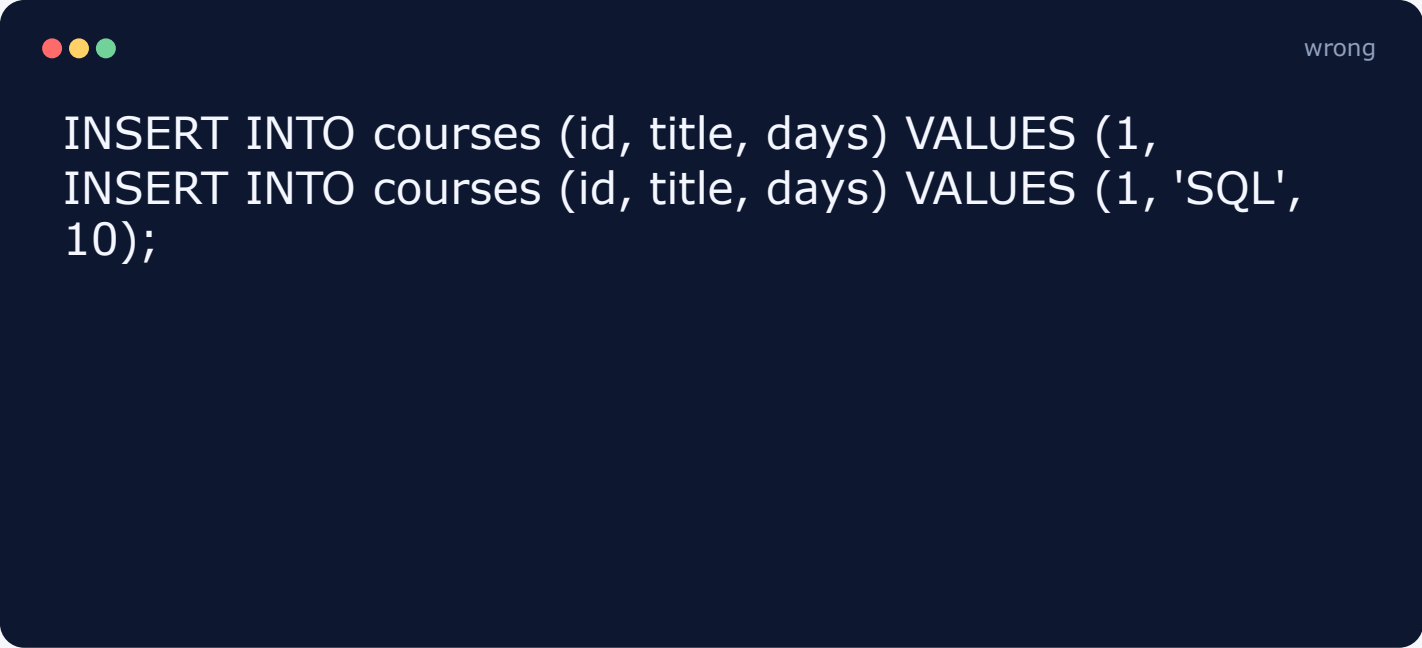
ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



```
INSERT INTO courses (id, title, days) VALUES (1,  
INSERT INTO courses (id, title, days) VALUES (1, 'SQL',  
10);
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Primary key должен быть уникальным. Второй INSERT повторяет id=1 и нарушает ограничение уникальности.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
INSERT INTO courses (id, title, days) VALUES (1, 'Python', 30);  
INSERT INTO courses (id, title, days) VALUES (2, 'SQL', 10);
```

fixed

ПРАВИЛО

**Не переписывай всё
решение. Найди
минимальное
осмысленное
исправление.**

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Спроектировать таблицу users с id, name и уникальным email.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

1 **Сущность — пользователь.**

2 id делаем primary key.

3 name обязателен.

4 email обязателен и уникален.

5 Проверяем пример INSERT.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY,  
  name TEXT NOT NULL,  
  email TEXT NOT NULL UNIQUE
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY,  
  name TEXT NOT NULL,  
  email TEXT NOT NULL UNIQUE  
);
```

```
INSERT INTO users VALUES (1, 'Anna', 'anna@example.com');
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

В users появляется одна валидная строка

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Спроектируй таблицу tasks: id, title, done и user_id.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

id — PRIMARY KEY, title — NOT NULL, done можно хранить как BOOLEAN, user_id станет ссылкой на пользователя.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
CREATE TABLE tasks (  
  id INTEGER PRIMARY KEY,  
  title TEXT NOT NULL,  
  done BOOLEAN NOT NULL DEFAULT FALSE,  
  user_id INTEGER NOT NULL  
);
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Таблица `tasks` готова для хранения задач пользователей

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Добавь foreign key из `tasks.user_id` в `users.id`.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
CREATE TABLE tasks (  
  id INTEGER PRIMARY KEY,  
  title TEXT NOT NULL,  
  user_id INTEGER NOT NULL,  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Каждая задача может ссылаться на существующего пользователя

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 Хранить несколько значений в одной колонке.

02 Не назначать первичный ключ.

03 Использовать NULL без понимания смысла отсутствия.

04 Дублировать данные вместо связи таблиц.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «database» и какую проблему это понятие помогает описать?

2

Чем «database» отличается от «column»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **database — организованное хранилище данных с правилами доступа.**

2 database: организованное хранилище данных с правилами доступа. column: именованное свойство записей таблицы.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 Создана таблица `courses` с тремя колонками

4 `Primary key` должен быть уникальным. Второй `INSERT` повторяет `id=1` и нарушает ограничение уникальности.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Спроектировать простую реляционную таблицу и понимать строки, колонки, первичный/внешний ключ, ограничения и NULL.
- ✓ Читать и объяснять пример темы «Базы данных и SQL».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

SQL-запросы, связи и индексы

Завтра: SQL-запросы, связи и индексы. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.